Lab Assignment 5: Regular Expression-Based Lexical Analyzer

Course: CS3104 - Compiler Design Laboratory

Date: 3/09/2026

Instructions

- Write your name and roll number on top of every source file as comments.
- Source files must be named exactly: `lexer.c`. The program must compile cleanly with plain gcc, e.g.: `gcc lexer.c -o lexer`

- No Lex/Flex or other lexer-generator tools are to be used — the objective of this lab is to implement the lexical-analysis logic using regular-expression-based token patterns.
- Submit only the source file through the same submission portal used for previous assignments. http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A lexical analyzer is the first phase of a compiler. It reads a source program as a sequence of characters and groups them into **lexemes**, assigning each lexeme a corresponding **token class**.In this assignment, you will implement a lexical analyzer for a small programming-language fragment using **regular expressions** to describe the valid forms of different tokens.

The analyzer must scan the input from left to right and recognize tokens belonging to the following categories:

Keywords:

```
if
else
while
int
string
return
```

Identifiers:

```
letter (letter | digit | _)*
```

Integer constants:

```
digit+
```

Real constants:

```
digit+ '.' digit+
```

Arithmetic operators:

```
+ - * /
```

Relational operators:

```
< <= > >= == !=
```

Assignment operator:

```
=
```

Delimiters:

```
( ) { } ; ,
```

Whitespace characters should be ignored.

The analyzer should produce the sequence of tokens corresponding to the input source program and report an appropriate **lexical error** whenever an unrecognized character or invalid lexeme is encountered.

Example

Input:

```
int sum = 10 + 20;
if (sum >= 25) return sum;
```

Expected output:

```
KEYWORD    int
IDENTIFIER sum
ASSIGN     =
INTEGER    10
PLUS       +
INTEGER    20
SEMICOLON  ;

KEYWORD    if
LPAREN     (
IDENTIFIER sum
GREATER_EQ >=
INTEGER    25
RPAREN     )
KEYWORD    return
IDENTIFIER sum
SEMICOLON  ;
```

```
ACCEPT
```

For an input such as:

```
int x = 10 @ 20;
```

the analyzer should report the invalid character:

```
KEYWORD   int
IDENTIFIER x
ASSIGN    =
INTEGER   10
LEXICAL ERROR: unexpected character '@'
```

and the program should end with: **REJECT**

Core Requirements

● **Regular-expression token definitions:** define regular expressions for each token category, including identifiers, integers, real numbers, operators, and delimiters.

● **Lexical analyzer (`lexer.c`):** read the input source program and scan it from left to right, producing the corresponding token stream.

● **Keyword recognition:** identifiers that exactly match one of the specified keywords must be reported as `KEYWORD` tokens rather than `IDENTIFIER` tokens.

● **Longest-match rule:** when more than one token pattern can match at the current position, select the token corresponding to the **longest possible lexeme**. For example:

```
>=
```

must be recognized as `GREATER_EQ`, not as `>` followed by `=`.

● **Token output:** for every recognized token, print its token class and lexeme.

● **Whitespace handling:** spaces, tabs, and newlines must be ignored and must not produce tokens.

● **Lexical error detection:** if no valid token pattern matches the current character, report a lexical error and identify the offending character.

● **Final verdict:** the program must end its output with a single line reading either: **ACCEPT** or **REJECT**

Implementation Guidelines

● Represent the token patterns using regular expressions rather than writing a separate character-by-character condition for every possible lexeme.

● Process the input sequentially from left to right. At each position, determine which token pattern matches the upcoming characters.

● Apply the **longest-match principle** before selecting the token. This is particularly important for operators such as:

```
=  ==
<  <=
>  >=
!  !=
```

● Keywords and identifiers use overlapping patterns. First recognize the lexeme as an identifier and then determine whether the resulting lexeme belongs to the keyword list.

● Integer and real-number patterns should be distinguished correctly. For example:

```
25     → INTEGER
25.50  → REAL
```

● The analyzer should continue scanning after recognizing a valid token until the complete input has been processed or a lexical error makes further scanning impossible.

● Keep lexical analysis separate from syntax analysis. The output of this assignment should be a **token stream**, which can later be consumed by a parser.

Compilation & Testing

```
$ gcc lexer.c -o lexer
$ ./lexer < input.txt
```

Test the analyzer using inputs containing:

1. Valid identifiers and keywords
2. Integer and real constants
3. Single-character operators
4. Multi-character relational operators
5. Different delimiters
6. Whitespace between tokens
7. Invalid characters
8. Lexemes that test the longest-match rule
9. Identifiers that contain keyword prefixes

For example:

```
ifx
```

must be recognized as:

```
IDENTIFIER ifx
```

and **not** as:

```
KEYWORD if
IDENTIFIER x
```

Deliverables

- `lexer.c`